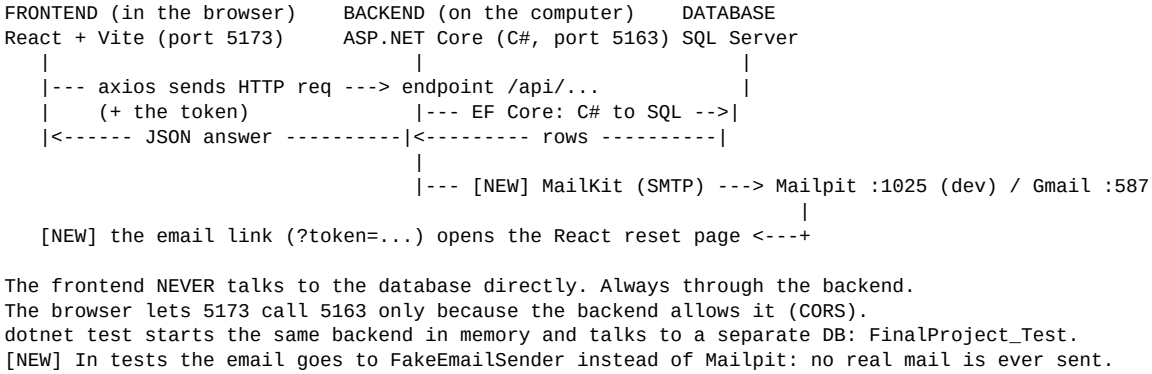
Track-System – Glossary

Last updated: 17 Oct 2026 (after step 6) | The words that keep coming back in this project, each one in plain language | **[NEW]** = added in step 6

1. The big picture – who talks to whom



The frontend NEVER talks to the database directly. Always through the backend.
The browser lets 5173 call 5163 only because the backend allows it (CORS).
dotnet test starts the same backend in memory and talks to a separate DB: FinalProject_Test.
[NEW] In tests the email goes to FakeEmailSender instead of Mailpit: no real mail is ever sent.

2. The platform words

Word	What it means
.NET	Microsoft's platform for running C# programs, like Node.js is for JavaScript. It includes the compiler, the libraries and the runtime. (dotnet --version)
ASP.NET Core	The part of .NET for building web servers and APIs. Our whole backend.
SDK / Runtime	SDK = the tools to build code. Runtime = only what is needed to run it. A developer needs the SDK. We installed the SDK.
Kestrel	The actual web server inside ASP.NET Core. It listens on the port and receives the requests. (The line "Now listening on: ...")
NuGet	The library store for .NET. Every PackageReference in the .csproj is downloaded from there. dotnet add package <name> downloads one and adds it to the .csproj.
Transitive package	A library we did not add ourselves; another library needs it and brings it in. Example: EF Core SqlServer -> Microsoft.Data.SqlClient -> System.Security.Cryptography.Pkcs. **[NEW]** Also MailKit -> MimeKit.
Direct reference wins	If the .csproj names a package version directly, that version is used instead of the transitive one. That is how we replaced the vulnerable Pkcs 9.0.11 with 10.0.13.
Vulnerability warning (NU1902 / NU1903)	A NuGet warning that a package version has a known security problem (moderate / high), with a GitHub advisory link. Fix: move to a patched version.
npm	The same idea for JavaScript. The frontend's package.json is its shopping list. (npm install)
package-lock.json	npm's exact list of installed versions. An npm install can rewrite it; if no package changed, undo it with git restore.
Vite	The tool that runs the React project while developing and builds it for production. Usually port 5173. (npm run dev)
.env.development / VITE_ prefix	The frontend's settings file. Only names that start with VITE_ reach the browser code, read as import.meta.env.VITE_API_URL. Not in git: each PC creates it.
Configuration	Everything .NET reads at startup, merged into one place: appsettings.json + User Secrets + appsettings.{Environment}.json. The code reads it with builder.Configuration["Jwt:Issuer"].
Section:Key (:)	The colon means "inside". Jwt:Key = the Key inside the Jwt section, whether it came from appsettings.json or from User Secrets.
Development environment	The mode set in launchSettings.json (ASPNETCORE_ENVIRONMENT). User Secrets are loaded only in Development.
Environment	The name of the mode the server runs in: Development (dotnet run), Testing and RateLimitTesting (our tests). Each one may have its own settings file. Try: dotnet run -- --environment Testing.

Word	What it means
appsettings.{Environment}.json	Loaded on top of appsettings.json and overrides only the keys it contains. appsettings.Testing.json swaps the DB to FinalProject_Test, the limit to 1000 **[NEW]** and MinResponseMs to 0.
[NEW] Override order	Later sources win: appsettings.json, then appsettings.{Environment}.json, then User Secrets (Development only). That is how one PC can send through Gmail while git keeps Mailpit.
[NEW] IConfiguration	The settings object itself. Program.cs reads it as builder.Configuration; a class asks for it in its constructor (SmtpEmailSender) and reads _config["Email:SmtpHost"].
[NEW] winget	Windows' own package manager: winget install axllent.mailpit installs Mailpit, winget list <id> shows whether it is installed.
[NEW] PATH	The list of folders Windows searches when you type a command. A program installed while VS Code was open is found only after VS Code is restarted ("is not recognized").

3. The web words

Word	What it means
HTTP request	A message the browser sends to a server: a method (GET/POST/...), an address, optional headers and optional body data.
GET / POST / PUT / DELETE	GET = give me data. POST = here is new data (for example login details). PUT = update something. DELETE = remove it.
Endpoint	One address on the server plus the code that answers it. Example: /api/health.
API	The whole collection of endpoints: the menu of everything the backend can do for the frontend.
REST	A common style for designing an API: addresses are nouns (/api/employees), methods are verbs.
JSON	The text format both sides speak, for example {"status":"ok"}.
Status code	A number describing the result: 200 OK, 201 created, 400 bad request, 401 not logged in, 403 not allowed, 404 address not found, 500 server crashed.
400 Bad Request (ours)	Wrong input. change-password answers 400 + a message (wrong current password, same password, broken rule). React shows the message and keeps the user logged in. **[NEW]** reset-password too: a bad link or a broken rule.
401 vs 403 (ours)	401 = no valid token (React deletes it and goes to Login). 403 = the token is fine but not allowed here, e.g. mustChangePassword "true" on /api/employees.
405 Method Not Allowed	The address exists but not for this method. Opening a POST-only endpoint in the browser (which sends GET) gives a blank page with 405.
423 Locked	Our answer for a locked account in step 4. No longer used: a locked account gets the same generic 401, so the lock stays silent.
429 Too Many Requests	The answer when one IP sends too many attempts: login, change-password, **[NEW]** forgot-password and reset-password share 10 per minute per IP, with a JSON message.
Header	A name: value line sent with a request or answer, outside the body. Example: Content-Type: application/json.
Authorization header	The standard header for credentials. We send: Authorization: Bearer <token>.
Bearer	"The bearer (holder) of this token". Whoever presents a valid token is treated as its owner, which is why a stolen token is dangerous until it expires.
WWW-Authenticate	A header the server adds to a 401 explaining why the token was refused, e.g. error="invalid_token", "The signature is invalid" or "The token expired".
Request body	The data part of a POST, for example {"eNum":1,"password":"..."}.
IP address / ::1	The caller's network address, saved in LoginLogs. ::1 is localhost in IPv6 (the same as 127.0.0.1). In tests it is NULL: the in-memory server has no network.
localhost	"This computer". Not reachable from outside, which is what we want while developing.
Port	The door number on the machine. 5163 = our API, 5173 = React, **[NEW]** 1025 = Mailpit's SMTP, 8025 = Mailpit's web inbox.
axios	A JavaScript library that sends HTTP requests from React. Our one shared copy is in src/api/client.js.

Word	What it means
baseURL	The address axios puts in front of every call, so pages only write "/api/...". Ours comes from VITE_API_URL.
Interceptor	An axios hook that runs on every request or answer. Ours: before each request add the Bearer token; after a 401 (not from login) remove the token and go to "/".
sessionStorage	Browser storage for one tab. The token lives there: it survives F5 and is gone when the tab closes.
CORS	A browser security rule: a page from port 5173 may not call port 5163 unless the server allows it. Policy "ReactApp" allows only the origins in Cors:AllowedOrigins.
Origin	Scheme + host + port of the page, e.g. http://localhost:5173. CORS rules are per origin.
Connection refused	The browser's "can't reach this page": nothing is listening on that address. The server is not running or the port is wrong.
camelCase JSON	ASP.NET Core writes C# names (FirstName) into JSON as firstName automatically.
Rate limiting	Limiting how many requests one IP may send in a time window. Built into .NET: AddRateLimiter + UseRateLimiter, our policy is called "auth".
favicon.ico 404	Browsers ask every site for a tab icon. Our API has none, so this red line in the Console is harmless.
[NEW] Query string (?token=)	The part of an address after ?, as name=value pairs. The reset link is /reset-password?token=<64 hex chars>.
[NEW] useSearchParams	A react-router hook that reads the query string: searchParams.get("token") gives the token, or null if the link has none.
[NEW] Fragment <> ... </>	Wraps several JSX elements without adding a real , because {token && (...)} may return only one element.
[NEW] disabled button	disabled={sending} stops double clicks while waiting; each extra click would send another email.
[NEW] try / catch / finally (JS)	finally runs after try or catch, whichever happened. Used to turn the button back on (setSending(false)).

4. The database words

Word	What it means
SQL Server	Microsoft's database program. SQL Express is the free local edition.
SSMS	SQL Server Management Studio, the app where you look at the tables by hand. F5 runs everything in the current tab; highlight lines first to run only them.
Connection string	One line saying which server, which database and how to log in. In appsettings.json.
Windows Authentication	Logging into the DB with your Windows user, so no password is written anywhere.
ORM	Object Relational Mapper: maps database rows to objects in code.
EF Core	Entity Framework Core, Microsoft's ORM. Translates our C# into SQL and rows back into objects.
DbContext	The EF Core class that represents the database. Every query goes through our AppDbContext.
DbSet	One line inside the DbContext representing one table. Ours: Employees, LoginLogs, **[NEW]** PasswordResetTokens.
Model / Entity	A plain C# class whose properties mirror the columns of one table (Employee.cs, LoginLog.cs, **[NEW]** PasswordResetToken.cs). EF Core fills it.
Migration	EF Core creating tables from C# classes. We do NOT use it: our tables are created by our own .sql scripts.
Provider	The package that knows one database's dialect. Ours: Microsoft.EntityFrameworkCore.SqlServer.
Primary key	The column that identifies each row uniquely (Employees.eNum, LoginLogs.logID, **[NEW]** PasswordResetTokens.tokenID).
IDENTITY	The DB fills the number automatically (1, 2, 3...). Numbers are never reused, even after a DELETE. A dropped and recreated DB starts again at 1.
NULL / NOT NULL	NULL = the column may be empty. NOT NULL = a value is required. **[NEW]** usedAt NULL = the link was never used.
DEFAULT	A value the DB fills in when none is given, for example role = 'Employee'.

Word	What it means
Foreign key	A rule that a column must point to an existing row in another table (Employees.teamNum -> Teams). A JOIN does not need it; it only enforces the link.
Log table without FK	LoginLogs.eNum has no foreign key on purpose: it stores the number that was typed, so attempts by unknown numbers are logged too and history survives a deletion.
[NEW] ON DELETE CASCADE	Part of a foreign key: deleting the employee deletes his reset-token rows too. Used on PasswordResetTokens.eNum, because a token only exists for a real employee.
[NEW] UNIQUE constraint / unique index	No two rows may have the same value. tokenHash is unique, so one hash finds exactly one row, and the search is fast.
[NEW] Named constraints (PK_ / FK_ / UQ_)	CONSTRAINT PK_PasswordResetTokens ... gives a rule a readable name instead of a random one, so errors and SSMS show which rule it is.
[NEW] char(64)	Text of exactly 64 characters. A SHA-256 hash in hex is always 64 characters long.
[NEW] datetime vs datetime2	Two SQL date types. Our real tables use datetime; EF builds datetime2 in the test DB. Both hold UTC values fine.
sys.foreign_keys	A system view listing every foreign key, used to find an auto-generated FK name.
[NEW] INFORMATION_SCHEMA.COLUMNS / sys.indexes	System views listing a table's columns and its indexes. We used them to check the new table in both DBs.
DROP CONSTRAINT	ALTER TABLE ... DROP CONSTRAINT [name] removes a rule (like an FK) without touching the data.
UPDATE ... SET ... WHERE	Changes existing rows. Always with WHERE, or every row changes.
UTC	World time with no time zone and no daylight saving. Israel is UTC+3 in summer, UTC+2 in winter. lockoutEnd [NEW] and the reset-token times are stored in UTC.
GETDATE() / GETUTCDATE()	SQL Server's local time / UTC time. Compare lockoutEnd with GETUTCDATE().
AT TIME ZONE	Converts a time for display: lockoutEnd AT TIME ZONE 'UTC' AT TIME ZONE 'Israel Standard Time'.
Empty result []	The query worked but the table has no rows.
Test database (FinalProject_Test)	A separate DB only for dotnet test, on the same server. It holds only the tables that have a DbSet, is dropped and rebuilt on every run, and may be deleted any time.
EnsureDeleted / EnsureCreated	EF Core methods: drop the whole database named in the connection string / create it with tables built from the DbSets. Used only for the test DB, behind a name check.
[NEW] Baseline schema script	Database/FinalProject_schema.sql: all tables as they were before the first change. A new PC runs it first, then every change script in order.
[NEW] Change script (Database/changes/)	One numbered .sql file per DB change (001_..., 002_...). In git, run once on each PC, never edited after it ran.
[NEW] Seed script	A script that adds test data (seed_test_employee.sql: one test employee). Dev only. IF NOT EXISTS makes it safe to run twice.
[NEW] GO	SSMS's batch separator: it runs everything above it before going on. Needed after USE and after CREATE TABLE.

5. The C# and code words

Word	What it means
Namespace	The address of a class so other files can find it with using .... Ours: TrackSystem.Api.Data, .Models, .Dtos, .Security, [NEW] .Services, and TrackSystem.Api.Tests.*.
using	A line at the top that lets the file see the classes of a namespace, e.g. using System.Security.Claims.
Class / Property	A class is a blueprint. Its properties are the fields inside it.
Inheritance (:)	class AppDbContext : DbContext receives all of DbContext's abilities. Also TestApiFactory : WebApplicationFactory<Program>.
[NEW] Interface	A contract: method signatures with no body. IEmailSender says "SendAsync(to, subject, htmlBody)" and nothing about how. Names start with I by convention.
[NEW] Implements (: IEmailSender)	A class that promises to have every method of the interface. SmtpEmailSender and FakeEmailSender both implement IEmailSender.

Word	What it means
Lambda (=>)	A small function written inline and run later, e.g. the code inside MapGet.
async / await	Waiting for the DB [NEW] or the mail server without freezing the server.
Task / async Task	The return type of an async method that gives back nothing. Our tests are async Task, so xUnit waits for them. Task<int> = gives back an int later.
[NEW] Task.Delay / Task.CompletedTask	Task.Delay(ms) waits without blocking the server (used for MinResponseMs). Task.CompletedTask = a Task that is already finished (the fake sender).
Dependency Injection	You ask for something in the parentheses and the server hands you a ready one (AppDbContext db, HttpContext http, ClaimsPrincipal user, [NEW] IEmailSender email, ILogger<Program> logger).
Service registration	Teaching the server how to build those objects (builder.Services...), before Build(). [NEW] builder.Services.AddScoped<IEmailSender, SmtpEmailSender>() = "whoever asks for IEmailSender gets a SmtpEmailSender".
[NEW] Lifetimes: Scoped / Singleton	Scoped = one object per request (AppDbContext, SmtpEmailSender). Singleton = one object for the whole app (the fake mailbox in tests).
Scope (CreateScope)	A short life for scoped services like AppDbContext. Each request gets one; test helpers create one with Services.CreateScope().
Middleware	Code every request passes through before the endpoint. Ours, in order: UseCors, UseRateLimiter, UseAuthentication, UseAuthorization.
Minimal API	Endpoints written directly in Program.cs (the older style is Controllers).
Attribute [ ]	A label in square brackets about the ONE thing right after it. Example: [Key].
[Key]	Tells EF Core which property is the primary key (ENum, LogID, [NEW] TokenID).
[NEW] [Index(..., IsUnique = true)]	An attribute on the class: EF builds a unique index on that property in the test DB (TokenHash).
[NEW] [Column(TypeName = "char(64)")]	Tells EF the exact SQL type. Without it EF would use nvarchar(max), which cannot have a unique index.
[NEW] Navigation property / [ForeignKey]	public Employee? Employee on PasswordResetToken: a link to the related row. [ForeignKey(nameof(ENum))] says which column holds the link, so EF knows the FK.
[NEW] Include(...)	Loads the related row in the same query: PasswordResetTokens.Include(t => t.Employee) gives the token and its employee together.
ENum vs enum	ENum is our property name; enum is a C# keyword. C# is case-sensitive, so they never clash.
Nullable ?	int? or string? means "may be empty": the C# side of a NULL column. Also for parameters: string? token may be null. [NEW] DateTime? UsedAt.
[NEW] Nullable warning CS8604	The compiler warns that a maybe-null value goes where null is not allowed. We fixed it with ?? throw (a clear error), not with ! (which only hides it).
= string.Empty	A starting value of "" so a required text property is never null.
Types mapping	nvarchar -> string, int -> int, bit -> bool, datetime -> DateTime, [NEW] char(64) -> string.
LINQ	C#'s query methods (Select, Where...). EF Core turns them into SQL.
.Select(...)	Chooses which fields to take, which is how the password never leaves the DB.
Anonymous object	new { ... } with no class name, used to shape the JSON output and the JSON bodies in tests.
ToListAsync()	Runs the query and returns the rows as a list.
FirstOrDefaultAsync(...)	Returns the first matching row, or null if none. Used to find the employee by eNum [NEW] and the token by its hash.
Results.Ok(...)	Returns HTTP 200 with the object as JSON.
Results.Json(obj, statusCode)	Returns any status code with a JSON body. Used for the generic 401 with a message.
Results.BadRequest / Results.Unauthorized	400 with a JSON message / an empty 401. Used in change-password, refresh [NEW] and reset-password.
[NEW] IResult / Task<IResult>	"Some HTTP answer" (what Results.Ok and Results.BadRequest return). The local function SameAnswer() returns Task<IResult> because it waits first.

Word	What it means
Results.Text(...)	Returns plain text (used once, in the temporary hash endpoint).
StatusCodes.Status401Unauthorized	Built-in named constants for the status numbers (401, 429...).
DTO	Data Transfer Object: a class describing exactly what goes in or out of an endpoint. Ours: LoginRequest, ChangePasswordRequest, **[NEW]** ForgotPasswordRequest, ResetPasswordRequest.
record	A short way to write a class that only holds data: public record LoginRequest(int ENum, string Password);
static class	A class you never create with new; you call its methods directly: PasswordRules.Validate(...), **[NEW]** ResetTokens.Create().
[NEW] Tuple ((string Token, string Hash))	Returns two named values at once. var (token, hash) = ResetTokens.Create(); unpacks them; _ skips a part you don't need.
[NEW] Raw interpolated string $""".."""	A multi-line text with {values} inside, used for the email's HTML body.
[NEW] try / catch	catch (Exception ex) catches an error so the code goes on. Used around sending the email: a mail error is logged and the answer stays the same.
[NEW] throw expression (?? throw)	x ?? throw new InvalidOperationException("...") stops with a clear message when a setting is missing (Jwt:Key, Email:SmtpHost, ResetPageUrl...).
[NEW] ILogger<Program>	ASP.NET's built-in logger. logger.LogError(ex, "... {ENum}", eNum) writes a red fail: line in the dotnet run terminal.
[NEW] Stopwatch	A precise timer: Stopwatch.StartNew(), then ElapsedMilliseconds. Used to pad forgot-password to the same total time.
Change tracking	EF Core remembers the original values of loaded rows. Changing a property marks the row as changed.
Add(...)	db.LoginLogs.Add(row) puts a new row on EF's "to save" list. Nothing is written yet.
SaveChangesAsync()	Writes everything on the "to save" list to SQL Server in one trip (UPDATEs and INSERTs together, all or nothing).
Local function	A small function written inside another one. AddLog(...) inside the login endpoint, CreateToken(employee) in Program.cs, **[NEW]** SameAnswer() inside forgot-password.
HttpContext	The object describing the current request: headers, caller's IP (http.Connection.RemoteIpAddress), user.
ClaimsPrincipal user	The caller as the token guard saw them. user.FindFirst("sub")?.Value gives the eNum as text.
int.TryParse(text, out var n)	Tries to turn text into a number: true + the number, or false (no crash). Used on the sub claim.
StringComparison.OrdinalIgnoreCase	Compares text ignoring capitals: "Active" equals "active". Used for Status.
?? (null-coalescing)	"If the left side is null, use the right side." Used to stop the server when Jwt:Key is missing.
?. (null-conditional)	"Only continue if not null." RemoteIpAddress?.ToString() gives null instead of crashing.
DateTime.UtcNow	The current time in UTC. Used for lockoutEnd, token expiry **[NEW]** and reset-link times.
TimeZoneInfo	Converts UTC to a time zone. "Asia/Jerusalem" gives Israel time for LoginLogs.
GetValue<int>(...)	Reads a setting and converts it to a number (ExpiresMinutes). GetValue("RateLimit:AuthPermitLimit", 10) also gives a fallback if the key is missing.
Method chaining	}) followed by .RequireAuthorization(); calls another method on the result, so the statement ends only after the last call.
Default / named parameters	CreateEmployeeAsync(string password, string status = "active", ...): skipped ones keep their default. status: "pending" sets just that one.
const	A value fixed when compiling, e.g. const string Password. Only constants may go inside attributes like [InlineData(Password, ...)].

6. The security words

Word	What it means
Hashing	Turning a password into scrambled text that cannot be turned back. We store and compare hashes only.
BCrypt	The hashing library we use for passwords. Deliberately slow, which makes guessing attacks expensive.
HashPassword / Verify	BCrypt.HashPassword(pwd) makes a 60-char hash. BCrypt.Verify(typed, hash) returns true/false by re-hashing the typed password with the salt stored in the hash.
$2a$11$...	The hash format: version ($2a$), work factor (11 = 2^11 rounds), 22 chars of salt, then the hash.
Work factor (cost)	How many rounds BCrypt runs. Higher = slower = harder to crack.
Salt	Random characters added before hashing, so the same password gives different hashes.
Lockout	After 5 wrong passwords the account is locked for 15 minutes (failedLoginAttempts, lockoutEnd). A correct login resets the counter. Silent: while locked, even the correct password gets the generic 401. **[NEW]** A password reset by email also unlocks.
Generic error message	The same 401 text for a wrong number and a wrong password, so nobody learns which employee numbers exist (user enumeration). Also for locked and not-active accounts.
Timing attack	Learning something from how long an answer takes. Login: unknown eNums and locked accounts run the same slow BCrypt check. **[NEW]** forgot-password: every answer waits until MinResponseMs.
Dummy hash	A real BCrypt hash of a throwaway text, made once at startup. Checked (result ignored) for unknown eNums and locked accounts, so every failed login takes about the same time.
Active-only login	A correct password still gets the generic 401 if Status is not "active" (pending / inactive). The wrong-password counter is not touched. **[NEW]** Same rule for forgot / reset.
Password rules	A new password needs 8-64 characters, a capital, a number and a special character (not a letter, digit or space). 64 because BCrypt ignores everything after 72 bytes.
Audit log	A record of events for later checks. LoginLogs: every attempt, with Israel time, IP and result.
Authentication	Who are you? The login, and the token check on every request.
Authorization	What are you allowed to do? The rules: which endpoint needs a token or a role.
JWT	JSON Web Token: a signed string the server gives after a successful login. React sends it back on every later request.
Header . Payload . Signature	The 3 parts of a JWT, separated by dots. Header = algorithm (HS256). Payload = the claims (readable by anyone). Signature = proof it was made with our key and not changed.
Tampered token	A token whose payload was edited (e.g. role Employee -> Admin) with the old signature kept. The server recomputes the signature, it does not match -> 401. Proven by a test.
Claim	One piece of data inside the token: a name and a text value.
sub	Subject: who the token belongs to. Ours: the eNum, e.g. "1".
role	What the user is (Employee / Manager / Admin). Used later for role rules.
mustChangePassword	Our own claim: "true" on first login. Such a token opens only change-password and refresh (403 elsewhere).
iss / aud	Issuer = who made the token (TrackSystem.Api). Audience = who it is for (TrackSystem.Client).
exp / iat / nbf	Expires at / issued at / not valid before, as seconds since 1/1/1970 (UTC).
Token expiry	Our tokens live 30 minutes (Jwt:ExpiresMinutes). After that: 401 and log in again, unless refresh renewed it.
Refresh endpoint / sliding session	While the user is active, session.js asks POST /api/auth/refresh for a new 30-minute token (at most once a minute). 30 minutes without activity -> logged out.
Idle logout	session.js checks every 15 s: token expired -> removed, back to Login, even without a click.
Signing key	The secret (64 random bytes, in User Secrets) that signs and checks tokens. Whoever has it can forge tokens. Tests make their own random key on every run.
Symmetric key / HMAC-SHA256 (HS256)	Symmetric = the same key signs and checks. HS256 = the signing method: a SHA-256 hash mixed with the key.

Word	What it means
SecurityTokenDescriptor	The "recipe" for one token: claims, issuer, audience, expiry, how to sign.
JsonWebTokenHandler	The "printer" that turns the recipe into the header.payload.signature text. ReadJsonWebToken does the reverse (decode only, no signature check).
Stateless	The server keeps no list of tokens. Each request carries its token and is checked on the spot. Logout = React deletes its copy. So an old token stays valid until it expires.
AddAuthentication / AddJwtBearer	Registers the token guard (the JwtBearer handler) and how it checks tokens.
TokenValidationParameters	The guard's checklist: signature, issuer, audience, lifetime, plus which claims are the name (sub) and the role.
ClockSkew	Allowed clock difference when checking expiry. Default 5 minutes; ours is 0, so 30 minutes means exactly 30.
MapInboundClaims = false	Stops .NET from renaming sub and role to old long URL names (a compatibility habit from older Microsoft login systems).
UseAuthentication	Middleware: reads the Bearer token on every request and identifies the caller. Blocks nothing by itself.
AddAuthorization / UseAuthorization	Registers and runs the rules system. Blocks only endpoints that have a rule.
Authorization policy	A named rule. Ours: the default policy (valid token AND mustChangePassword "false") and "PasswordChange" (any valid token).
RequireClaim	Part of a policy: the token must hold this claim with this value, else 403.
RequireAuthorization()	The rule on one endpoint. Empty = the default policy; with a name = that policy, e.g. RequireAuthorization("PasswordChange").
Fixed window / partition key	Our limiter counts requests per 1-minute window. Each IP (the partition key) has its own counter; no IP -> one shared "unknown" counter (as in tests).
RequireRateLimiting("auth")	Puts the "auth" limit on one endpoint: login, change-password, **[NEW]** forgot-password and reset-password share it.
User Secrets	Secret settings stored on your own PC, outside the project folder (AppData\Roaming\Microsoft\UserSecrets\<id>\secrets.json).
UserSecretsId	The name of the project's secrets folder, written in the .csproj. Not secret.
Never expose the hash	Even a hashed password is never sent to the browser.
jwt.io	A website that decodes a token so you can read its payload. Paste test tokens only, never the key.
[NEW] Reset token	A random one-time secret (not a JWT) that proves the person opened the email. 32 random bytes = 64 hex chars, valid 30 minutes, usable once.
[NEW] RandomNumberGenerator	The cryptographic random generator (RandomNumberGenerator.GetBytes(32)). Its output cannot be predicted, unlike new Random().
[NEW] Hex (ToHexStringLower)	Bytes written as 0-9 a-f, two characters per byte. Safe inside a URL with no escaping. Always lowercase, so the same token always gives the same text.
[NEW] SHA-256	A fast one-way hash: the same input always gives the same 64 hex chars. Right for reset tokens (random, cannot be guessed, the row must be FOUND by the hash); wrong for passwords (too fast to guess them).
[NEW] Store only the hash	The DB keeps SHA-256(token); the real token is only in the email. Someone who reads the DB still cannot reset anyone's password.
[NEW] One-time / replaced / expired link	usedAt is set when the link is used, or when a newer link replaces it (only the newest works). expiresAt = createdAt + 30 min. Any of these -> 400.
[NEW] Account enumeration (forgot password)	Learning which eNums exist from a different answer. forgot-password always gives the same 200 text, after the same minimum time.
[NEW] Minimum response time (MinResponseMs)	Every forgot-password answer waits until 1500 ms have passed since the start, so a real employee (email sent) and an unknown eNum take the same time. Tests set 0.
[NEW] HTML encoding (HtmlEncode)	Turns < > & " into safe text before putting a DB value (the first name) into the email's HTML, so it cannot change the email.

Word	What it means
[NEW] App Password (Gmail)	A 16-character password Google creates for one app only (needs 2-Step Verification). Goes into User Secrets as Email:Password; can be deleted any time.

7. Tools and workflow

Word	What it means
git add / commit / push	add = choose the changes. commit = save a snapshot with a message. push = upload to GitHub.
git status	Shows what changed before you commit. Check it every time: no secrets, no *.http files.
[NEW] git status -uall	Lists every untracked file one by one, not only the new folder.
git add -n <path>	Dry run: lists what git WOULD add, adds nothing. Good for checking a new folder.
git diff --stat <file>	How many lines changed in a file, without showing them.
git restore <file>	Throws away local changes to one file and puts back the committed version.
git log -1 --oneline	Shows the last commit in one line.
.gitignore	The list of files git must ignore: bin, obj, node_modules, secrets, *.http and .env.* (so .env.development too).
bin / obj	Folders .NET generates while building. Never edit them, never commit them.
Build vs Run	Build = compile. Run = start. dotnet run does both; dotnet build only checks it compiles.
Hot reload	The backend does not reload by itself: after every change, Ctrl+C and dotnet run again. **[NEW]** Vite (the frontend) does reload by itself on save.
Working directory	The folder the terminal is in. API commands run in Backend\TrackSystem.Api, test commands in Backend\TrackSystem.Api.Tests.
cd	Change directory, e.g. cd Backend\TrackSystem.Api. cd ..\.. goes up two folders.
MSB1003	dotnet found no .csproj in the current folder. Fix: cd into the project folder.
Script Table as	SSMS menu that writes the SQL of an existing table. Do not press F5 in that tab: it tries to create the table again ("There is already an object named...").
dotnet user-secrets init / set / list	init = create the project's secrets folder (once). set "Jwt:Key" "..." = save a secret. list = show them. **[NEW]** remove "Email:SmtpHost" = delete one.
dotnet new xunit	Creates a test project from the xUnit template (-n name, -o folder).
dotnet add reference	Links one project to another, e.g. the tests to TrackSystem.Api.
[NEW] dotnet add package <name>	Downloads a NuGet library and adds its PackageReference to the .csproj (dotnet add package MailKit).
dotnet test	Builds and runs every test, then prints total / failed / succeeded. --logger "console;verbosity=detailed" lists each test.
[NEW] Output through a pipe (| Select-String)	When dotnet test output goes through |, dotnet switches to a plainer format, so a filter may show nothing. Check without the pipe, or with Ctrl+F in the terminal.
dotnet nuget why <project> <package>	Shows which package brought in a transitive package.
dotnet list package --vulnerable --include-transitive	Lists packages with known security problems, including indirect ones.
dotnet run -- --environment Testing	Starts the API in another environment (the args after -- go to our program).
REST Client / .http file	A VS Code extension for sending requests from a text file. Click Send Request (not the URL, which opens the browser with GET). ### starts a new request.
# @name / {{login.response.body.token}}	REST Client variables: name a request, then reuse a field from its answer, like the token.
F12 -> Network	The browser's developer tools tab that shows each request and its status code. Application -> Session Storage shows the saved token.

Word	What it means
F12 -> Console context "top"	Console commands run in the page selected at the top. It must be the localhost page, not another frame.
npm install / "vite is not recognized"	npm install downloads the frontend libraries into node_modules. Without it, npm run dev cannot find Vite.
[NEW] Send-MailMessage	An old PowerShell command we used once to test Mailpit (-SmtpServer localhost -Port 1025 ...). The "obsolete" warning is harmless. Without parameters it asks for From / To.
[NEW] File name extensions (Windows)	File Explorer hides them by default, so a file may really be .sql.txt. View -> Show -> File name extensions. Creating files in VS Code avoids it.
[NEW] Git letters in VS Code (M / U)	M = modified, U = untracked (new). A changed file without M usually means it is not saved yet (white dot on its tab).

8. The testing words

Word	What it means
Automatic test	Code that checks other code and answers pass / fail. dotnet test runs all of them in seconds, every time, the same way.
Unit test	Tests one small piece alone, with no server and no DB. Ours: PasswordRulesTests, **[NEW]** ResetTokensTests.
Integration test	Tests the pieces together: real HTTP request -> Program.cs -> EF Core -> SQL Server. Ours: everything in the Api/ folder.
xUnit	The test framework we use. It finds the test methods, runs them and reports the result.
Test project	A separate project (TrackSystem.Api.Tests) that holds only tests and references the API project. It is never part of the real server.
[Fact]	Marks a method as one test with fixed data.
[Theory] + [InlineData]	One test method run once per InlineData row; the row values arrive as the method's parameters. Each row counts as a test.
Assert	The checks. Equal(expected, actual), Null / NotNull, True / False, Single (exactly one item, returns it), All (every item), InRange, Contains / DoesNotContain, NotEqual, **[NEW]** Empty, Matches (regex). A failed Assert stops the test.
[NEW] Assert.Equal(..., precision: 1)	Compares numbers rounded to 1 decimal, e.g. 30.0 minutes = 30.
Expected vs actual	Assert.Equal(expected, actual): expected first. On failure xUnit prints both values.
WebApplicationFactory<Program>	Microsoft's factory (package Microsoft.AspNetCore.Mvc.Testing) that starts our real Program.cs in memory, with no port, and gives HttpClients that call it.
Factory	An object whose job is to create other things. TestApiFactory creates the test API, the clients, test employees and tokens.
ConfigureWebHost / UseEnvironment / UseSetting	Where the factory changes the API before it starts: the environment ("Testing") and single settings (the random Jwt:Key).
[NEW] ConfigureTestServices	Runs AFTER Program.cs registered its services, so a test can replace one: RemoveAll<IEmailSender>() then AddSingleton<IEmailSender>(Emails).
[NEW] Fake / test double	A stand-in that behaves like the real thing for the test. FakeEmailSender sends nothing; it keeps every email in a list so the test can read the link.
[NEW] ConcurrentQueue	A list that is safe when two threads add at the same time. Holds the fake's sent emails.
[NEW] Regex in tests	Regex.Match(body, "token=([0-9a-f]{64})") pulls the token out of the email; Groups[1] is the part inside ( ).
[NEW] Known test value	A published input/output pair: SHA-256("abc") = ba7816bf... If our Hash() ever changes algorithm or letter case, the test fails.
[NEW] Simulating time	Instead of waiting 30 minutes, ExpireResetTokensAsync sets expiresAt in the past, then the test checks the link is refused.
CreateClient()	Gives an HttpClient whose requests go straight into the in-memory API.

Word	What it means
Fixture	An object xUnit creates once and shares with many tests, e.g. our factory.
IClassFixture<T>	One T shared by all tests of one class.
Collection / ICollectionFixture<T>	Classes marked [Collection("Api")] share ONE T and run one after another, never in parallel. Needed because they share one test DB **[NEW]** (and one fake mailbox).
IAsyncLifetime	Gives a fixture InitializeAsync (run once before its tests) and DisposeAsync (after). Our factory builds the test DB in InitializeAsync.
Safety guard	A check that stops the tests before doing damage: TestApiFactory refuses to drop any DB that is not FinalProject_Test.
Test isolation	Each test creates its own employee (random email / user name), so tests never depend on each other's data or order. **[NEW]** Emails are found by that unique address.
Control test	A test that should pass, next to one that should fail, to prove the failure has only one cause. Our own-key token -> 200 next to the other-key token -> 401.
EnsureSuccessStatusCode()	Throws if the answer is not 2xx, so a helper like LoginAsync fails the test right away with the status.
Base64UrlEncoder	Decodes / encodes token parts (URL-safe base64). Used to build the tampered token.
ITestOutputHelper	xUnit's way to print text from a test into the report (_output.WriteLine).

9. The email words (all new in step 6)

Word	What it means
[NEW] SMTP	Simple Mail Transfer Protocol: the language programs use to hand an email to a mail server (connect, log in, send, QUIT).
[NEW] MailKit	The standard .NET library for sending email over SMTP (NuGet package). Microsoft recommends it instead of the old built-in SmtpClient. Used only inside SmtpEmailSender.
[NEW] MimeKit	Comes with MailKit. Builds the email itself: MimeMessage, MailboxAddress (name + address), TextPart("html") for the body.
[NEW] Mailpit	A separate program on your PC that pretends to be a mail server: it accepts every email on port 1025 and shows it at http://localhost:8025. Nothing leaves the PC. Forgets its emails when closed.
[NEW] Mailpit vs MailKit	MailKit = the library in OUR code that sends. Mailpit = the fake post office that receives. Gmail = the real post office.
[NEW] StartTLS / SecureSocketOptions	StartTls = start plain, then switch to an encrypted connection (Gmail, port 587). None = no encryption (Mailpit only). Set by Email:UseStartTls.
[NEW] From address	Who the email says it is from (Email:FromName + Email:FromAddress). With Gmail it must be that Gmail address.
[NEW] Reset link	ResetPageUrl + ?token=... = http://localhost:5173/reset-password?token=<64 hex>. The React page sends the token back to reset-password.
[NEW] Same answer, nothing sent	Unknown, pending or inactive eNum: no row, no email, but the same 200 text after the same time.
[NEW] Mail error is logged, not shown	If Mailpit / Gmail is down, the endpoint catches the error, logs it in red, and still answers the same 200 (a 500 would reveal that the account exists).
[NEW] Test addresses you own	With real email the link goes to Employees.Email. Test employees must have addresses you own; .local addresses go nowhere and are safe for the repo.